质检和交付要求

Motion QC 质检程序手册

motion_qc_v0.1.0.zip

README.md 中的文档内容可能过时, 以本文档为准

基于视觉语言模型（VLM）的**视频帧质量检测工具**。
从视频生成缩略图 → 调用多模态模型分析 → 输出结构化质检结果。

轻量、模块化，适合集成到自动化流水线中。

功能特点

- 自动生成视频缩略图
- 调用多模态大模型进行画面分析
- 自动提取 JSON 结果

- 使用 Pydantic 结构化解析
- 模块解耦，易于扩展

核心流程

```
视频 → 缩略图 → 图像编码 → 构建Prompt → LLM分析 → JSON解析 → 结构化结果
```

快速使用

1. 安装依赖包

```
uv sync
```

或者

```
pip install -r requirements.txt
```

2. 进入 `src\motion_qc\vlm\config.py` 配置你的 VLM 供应商

```
llm_config: dict = {
    "general": {
        "model": general_model,
        "system_prompt": " ",
        "api_key": DASHSCOPE_API_KEY,
        "base_url": ALIYUN_BASE_URL,
    },
    "ollama": {
        "model": "qwen3.5:35b-a3b",
        "api_key": "any_string",
        "base_url": OLLAMA_BASE_URL,
```

```
    },
    "ucloud": {
        "model": "Qwen/Qwen3.5-35B-A3B-FP8",
        "api_key": "any_string",
        "base_url": UCLOUD_BASE_URL,
    },
}

```

VLM 模型推荐使用 `Qwen3.5-35B-A3B` ,在速度和效果上比较平衡

如果是本地部署,推荐在 RTX 4090 设备上使用 vLLM 部署 `Qwen/Qwen3.5-35B-A3B-FP8`

```
# RTX4090 48G 参考的部署参数
vllm serve Qwen/Qwen3-VL-30B-A3B-Instruct-FP8   --tensor-parallel-size 1   --mm-encoder-tp-mode data   --gpu-memory-utilization 0.90   --async-scheduling  --max-model-len 8196 --max-num-seqs 8
```

3. 运行示例代码

```
代码块
uv run scripts/check_single_video.py
```

或者

```
代码块
python scripts/check_single_video.py
```

3. 核心代码

```
代码块
from motion_qc.media.thumb import gen_video_thumb

from motion_qc.vlm.chatbot import ChatBot
from motion_qc.vlm.prompt import build_messages
from motion_qc.vlm.config import llm_config

```

```python
from motion_qc.types import FrameCheckResult

chatbot = ChatBot(llm_config.get("general", {}), silent=True)

# 生成缩略图
img_path = gen_video_thumb(
    video_path="input.mp4",
    thumbnail_path="thumb.jpg",
    width=720,
)

# 构建请求
img_b64 = chatbot.encode_image(img_path.as_posix())
msgs = build_messages(img_b64)

# 获取结果
resp = chatbot.chat_with_messages(msgs)
resp_json = chatbot.extract_json(resp)

result = FrameCheckResult.from_json(resp_json)
print(result)
```

目录结构

代码块

```
src/motionx_qa
├── media/      # 视频缩略图生成
├── vlm/        # 模型调用与 prompt
└── types.py    # 结构化结果定义
```

适用场景

- 视频自动审核
- 画面质量检测
- 批量视频数据分析

交付要求概述

- 每完成一个批次的交付，需要把该批次的审核结果整理成一张结构固定的表 (如下表)
- 这张表需要写入一个 `sqlite3` 数据库文件，并与本批次的通过文件一起上传到对象存储
- 数据库文件命名规则： `{批次名}.db`
  - 例：批次 `b2_20260303` 的数据库文件为 `b2_20260303.db`
  - 批次名规则详细, 参考 S3 对象存储 数据上传手册

image	passed	problems	comment
b2_20260303/体育1/Aerobic_Gymnastics/102078_cNhgeTOt-9I_21/102078_cNhgeTOt-9I01.mp4	pass		画面中仅有一名主要人物 见，处于健身运动状态， 多人干扰，符合采集标准。
b2_20260303/体育1/Aerobic_Gymnastics/108319_t9pVPtgvlAc_2282/108319_t9pVPtgvlAc01.mp4	fail	["no_full_body","other"]	画面中未出现人物主体， 名单信息，无法满足全身 动动作的采集要求。
b2_20260303/体育1/Aerobic_Gymnastics/108319_t9pVPtgvlAc_2282/108319_t9pVPtgvlAc02.mp4	fail	["no_full_body"]	画面中未出现人物主体， 运动情况，不满足单人全 要求。

表结构与字段含义

- 表中每一行对应一个视频的审核结果
- 字段要求：
  - `image`：填写**本批次的相对路径**
  - 其他字段：填写 **VLM 的审核结果输出**（按你们现有的审核产物字段原样落库即可）

落库规范

- 数据库中包含一张主表, 名称为 `results`）
- 主键用自增 `id`，并对 `image` 建唯一索引（避免重复）
- **入库前处理**
  - 去掉所有标记为 `fail` 的记录（即数据库里不应出现 `fail` 行）
- **上传文件规则**
  - 只上传标记为 `pass` 的文件
  - 建议上传前人工复核一遍 `pass` 文件列表（避免漏传）

产物清单

- 对象存储中该批次应包含：
  - `{批次名}.db` （SQLite 数据库文件）
  - 所有 `pass` 的文件（与 `image` 字段相对路径一致）